

Tarea 1: Computación de Alto Rendimiento (IIC3533)

Josefina Abbott Cerda, José Ignacio Salas Coeymans y Borja Smith Vergara

11 de septiembre de 2026

(a) Generación de datos sintéticos

Para generar los datos sintéticos, seguimos los siguientes pasos:

1. Generar los datos, usando el generador aleatorio de NumPy (`np.random.default_rng`) fijando una semilla base, nosotros la establecimos como 1111.
2. Generar el vector de coeficientes reales $\beta^* \in \mathbb{R}^{301}$ muestreando de una distribución normal estándar $\mathcal{N}(0, 1)$.
3. Crear una matriz base de $100\,000 \times 300$ con valores independientes $\mathcal{N}(0, 1)$.
4. Concatenar horizontalmente un vector columna de unos a la matriz base para incorporar el intercepto (β_0), obteniendo así la matriz de diseño $X \in \mathbb{R}^{100\,000 \times 301}$.
5. Calcular el vector de respuesta $y \in \mathbb{R}^{100\,000}$ con el producto matricial $y = X\beta^* + \varepsilon$, donde ε es un vector de ruido con 100 000 muestras i.i.d. de $\mathcal{N}(0, 1)$.

Para correr el script de generación de datos, se puede ejecutar el siguiente comando en la terminal:

```
python src/data_generation.py
```

(b) Implementaciones de bootstrapping

Evolución de `bs_auto.py`

1. Cambios implementados por versión

- **Versión 1 (Ingenua):** Utiliza la configuración por defecto de scikit-learn. Emplea `LinearRegression()` pese a que X ya contiene una columna de unos; además, al guardar solamente `coef_`, pierde el intercepto separado del modelo. Por ello esta versión sirve como punto de partida, pero no entrega correctamente el intervalo de β_0 .
- **Versión 2:** Introduce `fit_intercept=False`. Dado que la matriz de diseño X ya incluye una columna de unos pre-concatenada, esto evita el cálculo de un intercepto redundante.

- **Versión 3:** Reemplaza el ciclo `for` por una comprensión de listas para expresar de forma más compacta la extracción de coeficientes. Este cambio no vectoriza los ajustes ni garantiza por sí solo una mejora medible.
- **Versión 4:** Declara explícitamente `bootstrap=True` (aunque ya es el valor por defecto) y fija `random_state=1234`, garantizando muestreo con reemplazo y reproducibilidad.
- **Versión 5:** Mantiene los parámetros óptimos del modelo, pero revierte la extracción de coeficientes al ciclo `for` con `.append()` para contrastar el manejo de memoria final.

2. Tiempos de ejecución

Versión	Características Principales	Tiempo ($p = 1$)
v1	Intercepto activo, ciclo <code>for</code>	66.040 s
v2	Sin intercepto, ciclo <code>for</code>	84.483 s
v3	Sin intercepto, list comprehension	118.939 s
v4	Sin inter., reproducible, bootstrap explícito	105.072 s
v5	Sin inter., ciclo <code>for</code> , bootstrap explícito	93.132 s

Cuadro 1: Tiempos de ejecución empíricos para distintas iteraciones de `bs_auto.py`.

3. Análisis de la versión óptima

La **Versión 4** es computacional y estructuralmente la más óptima.

En primer lugar, al definir `fit_intercept=False`, se evita estimar un intercepto separado cuando la matriz X ya incluye la columna de unos. Declarar `bootstrap=True` hace explícito el remuestreo exigido, mientras que `random_state` permite reproducir los mismos intervalos al cambiar el número de procesos.

Los tiempos variaron entre 66 y 119 s y no siguen una tendencia monótona; una sola medición por versión no permite atribuir esas diferencias a la forma de extraer solo 48 vectores. La v1 fue la más rápida, pero es incorrecta para β_0 . Elegimos la v4 por claridad, correctitud y reproducibilidad, no por ser el mínimo aislado. Los valores se guardaron en `results/computer_1/timings_b.csv`.

Evolución de `bs_sklearn.py`

1. Cambios implementados por versión

- **Versión 1 (Ingenua):** Utiliza el generador aleatorio global (`np.random.choice`) dentro de cada proceso trabajador. Su estado no queda asignado explícitamente a cada remuestreo y, dependiendo de cómo se creen los procesos, puede producir secuencias duplicadas o resultados no reproducibles. Además, mantiene activado el cálculo del intercepto por defecto.
- **Versión 2:** Aísla la generación de números aleatorios asignando una semilla determinista única (`seed_b`) a cada proceso mediante `np.random.default_rng(seed_b)`.

Esto incrementa levemente el tiempo de ejecución por el costo de instanciar 48 generadores distintos, pero es matemáticamente obligatorio para garantizar la validez estadística.

- **Versión 3:** Introduce `fit_intercept=False`. Al igual que en `bs_auto.py`, esto elimina el procesamiento redundante de la matriz, reduciendo los tiempos de ejecución sin cambiar el vector de coeficientes retornado a los procesos trabajadores.

2. Tiempos de ejecución

Versión	Características Principales	Tiempo ($p = 1$)
v1	RNG global (no reproducible), intercepto activo	87.877 s
v2	RNG independiente, intercepto activo	87.005 s
v3	RNG independiente, sin intercepto	71.948 s

Cuadro 2: Tiempos de ejecución empíricos para distintas iteraciones de `bs_sklearn.py`.

3. Análisis de la versión óptima

La **Versión 3** representa la implementación correcta y más eficiente utilizando scikit-learn de forma explícita. El manejo independiente de las semillas hace reproducibles los remuestreos, mientras que la desactivación del intercepto evita trabajo redundante y redujo el tiempo observado a 71.948 s.

Evolución de `bs_numpy.py`

1. Cambios implementados por versión

- **Versión 1 (Ingenua):** Traduce literalmente la fórmula de Mínimos Cuadrados Ordinarios entregada en el enunciado: $\hat{\beta} = (X^T X)^{-1} X^T y$. Utiliza `np.linalg.inv()` para calcular explícitamente la matriz inversa antes de realizar las multiplicaciones matriciales.
- **Versión 2:** Reemplaza la inversión explícita reescribiendo el problema como la resolución de un sistema de ecuaciones lineales: $(X^T X)\hat{\beta} = X^T y$. Utiliza `np.linalg.solve()` para encontrar los coeficientes de manera directa.

2. Tiempos de ejecución

Versión	Características Principales	Tiempo ($p = 1$)
v1	Cálculo explícito con <code>np.linalg.inv()</code>	5.480 s
v2	Resolución de sistema con <code>np.linalg.solve()</code>	5.566 s

Cuadro 3: Tiempos de ejecución empíricos para distintas iteraciones de `bs_numpy.py`.

3. Análisis de la versión óptima

La **Versión 2** es la implementación óptima y definitiva para esta sección.

En esta ejecución aislada, `inv` fue 0.086 s más rápido, una diferencia demasiado pequeña para establecer una ventaja. Elegimos `solve` porque evita construir explícitamente la inversa y es la formulación numérica apropiada para resolver el sistema.

Al utilizar `np.linalg.solve()`, NumPy delega la operación a subrutinas optimizadas de álgebra lineal (LAPACK), que resuelven el sistema $(X^\top X)\hat{\beta} = X^\top y$ sin construir explícitamente la inversa. Esto reduce operaciones y error numérico respecto de `inv`; no obstante, ambas versiones forman las ecuaciones normales y, por tanto, siguen siendo menos estables que resolver directamente el problema de mínimos cuadrados mediante QR o SVD.

Para correr las implementaciones, se pueden ejecutar los siguientes comandos en la terminal:

```
python src/bs_auto.py
python src/bs_sklearn.py
python src/bs_numpy.py
```

(c) Correctitud y reproducibilidad

Primero calculamos $\hat{\beta}$ sobre el conjunto completo, como indica el paso 1 del algoritmo. El error relativo $\|\hat{\beta} - \beta^*\|_2 / \|\beta^*\|_2$ fue 0.00311, lo que confirma que el ajuste recupera adecuadamente los coeficientes verdaderos.

Luego ejecutamos cada versión dos veces con $p = 1$ y una vez con $p = 4$, usando las mismas semillas. La Tabla 4 resume la cobertura, el ancho medio de los intervalos y la máxima diferencia entre ejecuciones.

Implementación	Cobertura	Ancho medio	Rep. $p = 1$	$p = 1$ vs. $p = 4$
BaggingRegressor	91.69 %	0.011282	0	0
sklearn + joblib	93.36 %	0.011373	0	0
NumPy + joblib	93.36 %	0.011373	0	0

Cuadro 4: Correctitud y reproducibilidad. Las dos últimas columnas muestran la máxima diferencia absoluta entre extremos de los intervalos.

Las coberturas son similares y razonablemente cercanas al 95 % nominal, considerando que solo se usan $B = 48$ remuestreos. Las versiones manuales emplean exactamente los mismos índices y sus intervalos difieren como máximo en $4,80 \times 10^{-14}$, debido al redondeo de las rutinas de álgebra lineal. **BaggingRegressor** usa otra secuencia de remuestreo y por eso no produce los mismos extremos, pero su cobertura y ancho son comparables.

La reproducibilidad requiere fijar la semilla de los datos, asignar una semilla determinista a cada remuestreo y fijar `random_state` en **BaggingRegressor**. Como la semilla depende de b y no del worker que ejecuta la tarea, cambiar p no cambia los intervalos. Los resultados completos se almacenan en `results/computer_1/correctness_c.csv`.

Para verificar la correctitud y reproducibilidad de las implementaciones, se puede ejecutar el siguiente comando en la terminal:

```
python src/correctness.py
```

(d) Backend multiprocessing de joblib

Creación de procesos y asignación de memoria: `Parallel` utiliza por defecto el backend basado en procesos `loky`. Este crea intérpretes Python separados, cada uno con su propio espacio de direcciones; por tanto, no debe asumirse que todos los objetos se comparten automáticamente mediante *fork*. En sistemas que crean procesos con *fork* puede existir inicialmente *Copy-on-Write*, pero `loky` también serializa argumentos para enviarlos a los workers y su comportamiento depende del sistema operativo.

Comportamiento de los arreglos X e y : Como X supera el umbral de tamaño por defecto de joblib, `loky` puede volcarlo automáticamente a un archivo temporal y abrirlo como *memmap* de solo lectura en los workers. Esto evita serializar una copia completa de X por tarea. El arreglo y es menor y puede serializarse normalmente. Sin embargo, la indexación avanzada $X[\text{indices}]$ y $y[\text{indices}]$ sí crea arreglos nuevos $X^{(b)}$ e $y^{(b)}$ dentro de cada tarea. Por tanto, se comparte o mapea la entrada original, pero cada worker necesita memoria adicional para sus remuestreos y matrices intermedias.

(e) Actividad del sistema y oversubscription

Los experimentos de este inciso se realizaron en un MacBook Air con Apple M1, 8 cores (4 de rendimiento y 4 de eficiencia), 8 GB de RAM y macOS (arquitectura ARM64). Se utilizó NumPy 2.0.2, joblib 1.5.3 y `threadpoolctl` 3.6.0. La configuración de NumPy indica que las operaciones BLAS y LAPACK están enlazadas con el framework Apple Accelerate.

Ejecutamos `bs_numpy.py` tres veces con $p \in \{1, 2, 4, 8\}$. Se observaron exactamente 1, 2, 4 y 8 procesos trabajadores, respectivamente. En una ejecución prolongada con $p = 8$, los workers registraron aproximadamente entre 53 % y 66 % de CPU cada uno, confirmando que los remuestreos se distribuyen entre procesos independientes.

p	Workers observados	Tiempo mediano [s]
1	1	5.052
2	2	4.409
4	4	4.003
8	8	5.531

Cuadro 5: Actividad observada al ejecutar la versión NumPy.

Ejecutamos `threadpool_info()` dentro de cada worker, después de cargar NumPy. En todos los casos retornó una lista vacía. Esta observación es coherente con que el NumPy instalado usa Apple Accelerate, cuyo pool interno no es reportado por `threadpoolctl`. Al cargar SciPy y scikit-learn sí aparecen un pool OpenBLAS y uno OpenMP de 8 threads, pero estos pertenecen a dichas bibliotecas y no demuestran que `np.linalg.solve` use esos pools.

El tiempo mejora hasta $p = 4$, pero empeora un 38 % al pasar de $p = 4$ a $p = 8$. Esto evidencia saturación y competencia por memoria; también es compatible con *oversubscription*, aunque la falta de visibilidad sobre Accelerate impide conocer el valor efectivo de t y atribuirle por sí sola la degradación.

En general, sí existiría oversubscription si cada uno de los p procesos lanzara t threads BLAS y se cumpliera $pt > p_{\text{máx}}$. En una instalación compatible con `threadpoolctl` puede corregirse envolviendo el cálculo de cada worker con `threadpool_limits(limits=1)` o configurando `inner_max_num_threads=1` en el backend `loky`. Para Apple Accelerate puede fijarse `VECLIB_MAXIMUM_THREADS=1` antes de iniciar Python. Las observaciones completas quedaron almacenadas en `results/computer_1/observations_e.csv`.

(f) Tiempos de ejecución en función de p

El primer computador posee $p_{\text{máx}} = 8$ cores lógicos. Comparamos las versiones finales `bootstrap_auto_v4`, `bootstrap_sklearn_v3` y `bootstrap_numpy_v2` usando los mismos datos ($N = 100\,000$, $k = 300$), $B = 48$ y las mismas semillas. Para cada implementación y cada $p \in \{1, \dots, 8\}$ se realizaron tres ejecuciones, cada una en un intérprete Python nuevo para impedir que `joblib` reutilizara un pool creado por una medición anterior. El orden de las 72 configuraciones fue aleatorizado para reducir el sesgo causado por calentamiento del equipo o por el orden de ejecución. Reportamos la mediana, por ser menos sensible que la media a interferencias transitorias del sistema.

p	BaggingRegressor	sklearn + joblib	NumPy + joblib
1	86.843	84.737	6.017
2	56.377	60.699	5.029
3	50.298	55.099	4.232
4	56.254	55.999	5.497
5	46.518	59.245	4.357
6	62.251	72.303	5.479
7	64.378	114.517	5.039
8	73.594	127.574	4.288

Cuadro 6: Mediana de los tiempos de ejecución $T(p)$, en segundos, para tres repeticiones aisladas en el primer computador.

La implementación basada solamente en NumPy fue la más rápida para todos los valores de p : con un proceso tardó aproximadamente 6.017 s, frente a 86.843 s de `BaggingRegressor` y 84.737 s de la versión manual con `scikit-learn`. La diferencia se debe principalmente a que `LinearRegression` realiza más trabajo general para ajustar cada modelo, mientras que la versión NumPy resuelve directamente las ecuaciones normales.

Las dos implementaciones basadas en `scikit-learn` mejoraron al aumentar p hasta un punto intermedio. El mejor tiempo mediano fue 46.518 s con $p = 5$ para `BaggingRegressor` y 55.099 s con $p = 3$ para `sklearn + joblib`. Con $p = 8$, `scikit-learn` empeoró hasta 127.574 s, debido a la presión de memoria de los remuestreos de 100 000 filas en un equipo con 8 GB. NumPy también obtuvo beneficio paralelo: su mejor tiempo fue 4.232 s con $p = 3$, aunque $p = 5$ y $p = 8$ entregaron tiempos cercanos. La mejora está lejos de ser lineal por el overhead, las copias de cada remuestra y la competencia por ancho de banda de memoria.

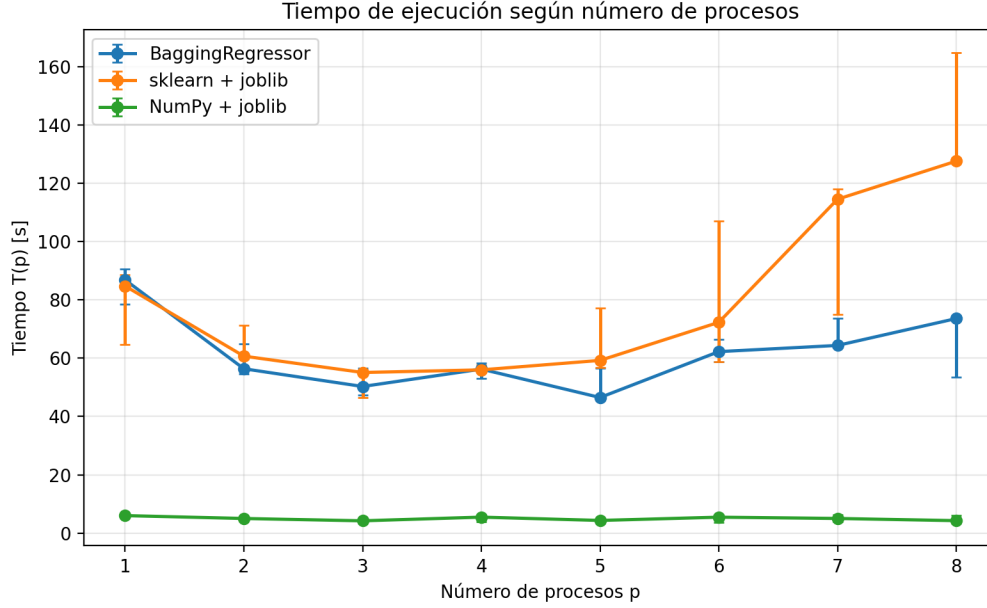


Figura 1: Tiempo mediano de ejecución en función de p . Las barras se extienden entre el mínimo y el máximo de las tres repeticiones.

(g) Speedup $S(p)$ y Eficiencia $E(p)$

A partir de las medianas de la Tabla 6, calculamos para cada implementación:

$$S(p) = \frac{T(1)}{T(p)}, \quad E(p) = \frac{S(p)}{p}.$$

Elegimos como $T(1)$ la mediana de las tres ejecuciones de la misma implementación con $p = 1$: 86.843 s para **BaggingRegressor**, 84.737 s para **sklearn + joblib** y 6.017 s para **NumPy + joblib**. Esta elección mantiene constantes el algoritmo, los datos, las semillas y la instrumentación, por lo que mide escalabilidad fuerte sin mezclar el resultado con una implementación secuencial diferente. Además, usar la mediana evita que una ejecución excepcionalmente lenta o rápida determine toda la curva.

p	BaggingRegressor		sklearn + joblib		NumPy + joblib	
	$S(p)$	$E(p)$	$S(p)$	$E(p)$	$S(p)$	$E(p)$
1	1.000	1.000	1.000	1.000	1.000	1.000
2	1.540	0.770	1.396	0.698	1.196	0.598
3	1.727	0.576	1.538	0.513	1.422	0.474
4	1.544	0.386	1.513	0.378	1.095	0.274
5	1.867	0.373	1.430	0.286	1.381	0.276
6	1.395	0.233	1.172	0.195	1.098	0.183
7	1.349	0.193	0.740	0.106	1.194	0.171
8	1.180	0.148	0.664	0.083	1.403	0.175

Cuadro 7: Speedup y eficiencia calculados a partir de los tiempos medianos.

Ninguna implementación alcanzó la curva ideal. **BaggingRegressor** obtuvo su mayor speedup en $p = 5$, con $S(5) = 1,867$ y $E(5) = 0,373$. La versión manual con scikit-learn alcanzó

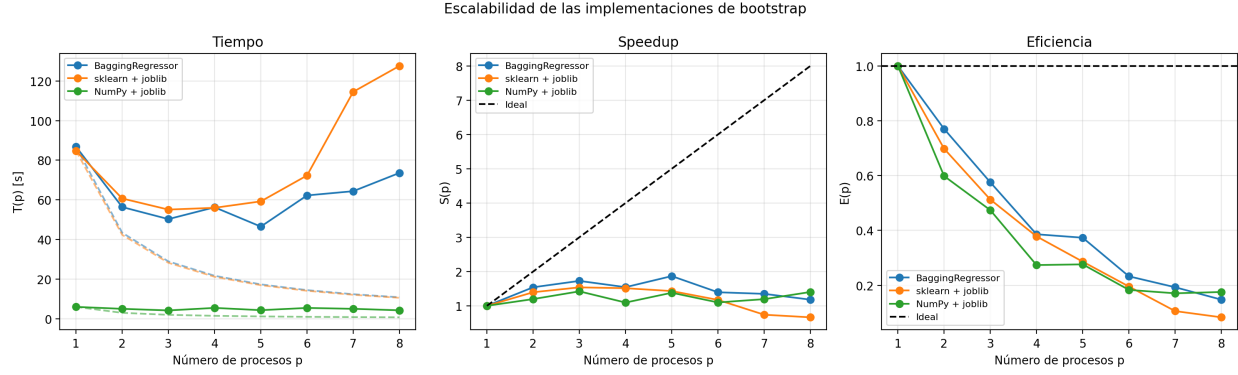


Figura 2: Tiempo, speedup y eficiencia observados. En el panel de tiempo, las líneas discontinuas corresponden a $T(1)/p$ para cada implementación; en los otros paneles, la curva ideal es $S(p) = p$ y $E(p) = 1$.

su máximo en $p = 3$, con $S(3) = 1,538$ y $E(3) = 0,513$. En ambas versiones el trabajo de ajustar 48 regresiones es suficiente para amortizar parte del costo de los procesos, pero la eficiencia disminuye al crecer p debido a creación y coordinación de procesos, distribución de tareas, movimiento de resultados y competencia por memoria.

La versión NumPy es la más rápida en tiempo absoluto y, con el problema más grande, obtiene speedup para todos los valores $p > 1$. Su máximo es $S(3) = 1,422$, con $E(3) = 0,474$. El speedup vuelve a 1.403 en $p = 8$, pero con una eficiencia de solo 0.175; por ello $p = 3$ es preferible: alcanza el menor tiempo usando menos procesos para $B = 48$, $N = 100\,000$ y $k = 300$.

(h) Overhead $T_o(p)$

Definimos el overhead total como

$$T_o(p) = pT(p) - T(1),$$

que mide el tiempo-procesador adicional respecto de la ejecución con un solo proceso. Por definición, $T_o(1) = 0$.

El overhead crece en general con p y explica la separación respecto de la curva ideal. Sus fuentes principales son el arranque de intérpretes, la serialización o mapeo de X e y , el despacho de las 48 tareas, la creación de $X^{(b)}$ e $y^{(b)}$, la recolección de coeficientes, la sincronización y la competencia por memoria y caché. En NumPy el overhead absoluto es menor, pero es grande respecto de su $T(1)$; por eso no obtiene speedup. Las pequeñas irregularidades menores corresponden a variabilidad experimental. El gran aumento de scikit-learn en $p = 7$ y $p = 8$ es consistente con la competencia por memoria al mantener varias remuestras de aproximadamente 241 MB.

p	BaggingRegressor	sklearn + joblib	NumPy + joblib
1	0.000	0.000	0.000
2	25.910	36.662	4.041
3	64.050	80.560	6.679
4	138.173	139.259	15.970
5	145.748	211.488	15.768
6	286.661	349.083	26.855
7	363.803	716.886	29.256
8	501.910	935.855	28.287

Cuadro 8: Overhead total $T_o(p)$, en segundos.

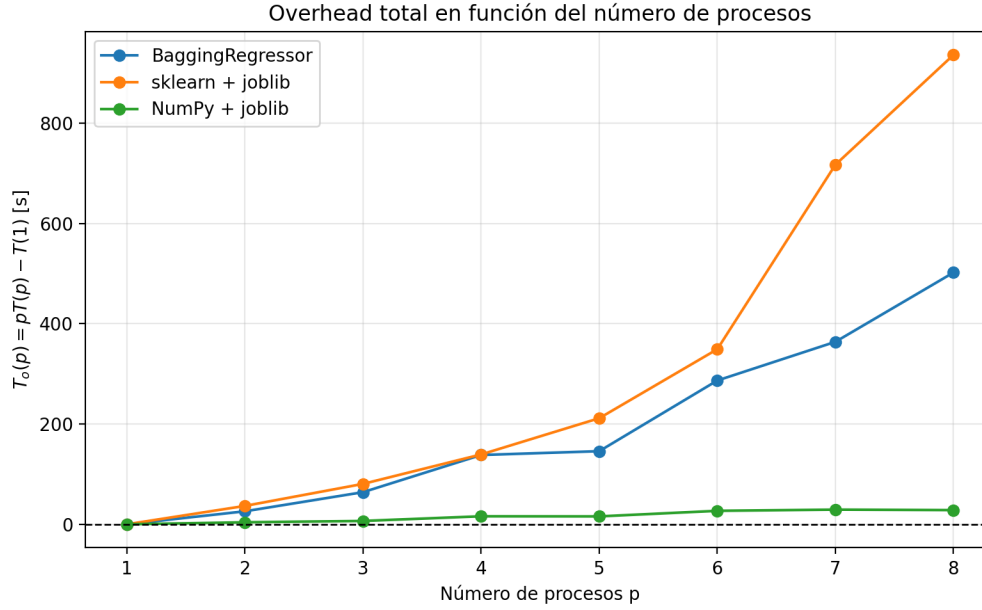


Figura 3: Overhead total en función del número de procesos.

(i) Variación de procesos y threads internos

Usamos la versión NumPy, que fue la más rápida en (b) y (f), y evaluamos las 20 combinaciones enteras (p, t) que satisfacen $pt \leq 8$. El límite se aplicó dentro de cada tarea mediante `threadpool_limits(limits=t)`. Cada combinación se ejecutó tres veces en un intérprete nuevo y se reportó la mediana.

La menor mediana observada fue 4.068 s con $(p, t) = (3, 2)$, seguida por 4.234 s con $(4, 1)$ y 4.436 s con $(3, 1)$. La configuración $(1, 1)$ tardó 5.580 s, por lo que el aumento de trabajo por remuestreo permite ahora amortizar el overhead de varios procesos. En cambio, $(8, 1)$ tardó 5.878 s y no mejora la ejecución secuencial de manera robusta.

Existe una limitación importante: `threadpool_info()` no detecta Apple Accelerate, por lo que t es el límite solicitado, pero no fue posible verificar el número efectivo de threads internos. En consecuencia, la mejor combinación nominal es $(3, 2)$, pero no es posible atribuir

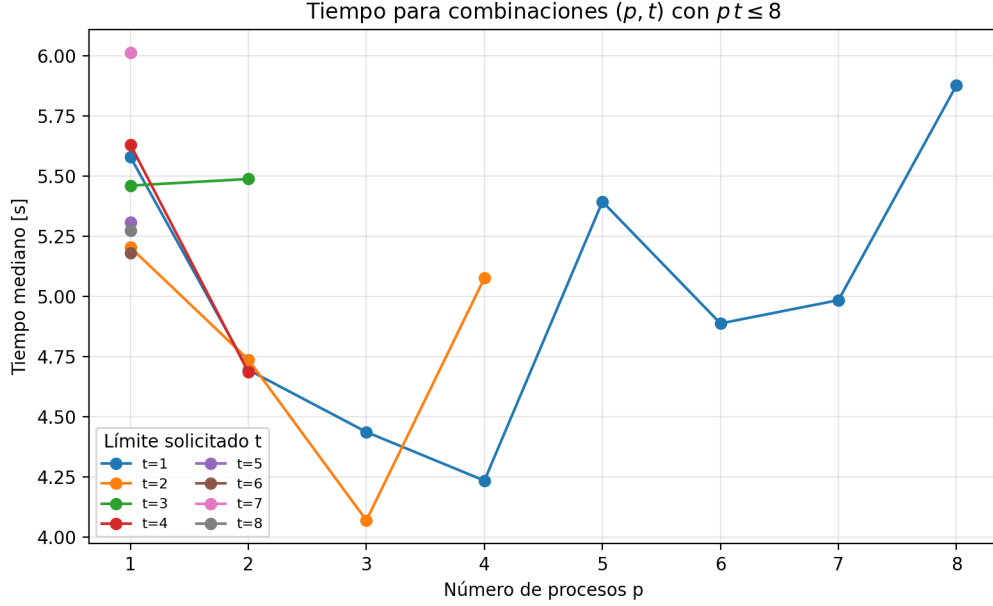


Figura 4: Tiempo mediano para todas las combinaciones (p, t) con $pt \leq 8$.

separadamente la mejora a dos threads internos efectivos. Aun así, esta combinación satisface $pt = 6 \leq 8$, obtiene la menor mediana y evita la saturación observada al usar ocho procesos; por ello es la recomendada para este computador. Los 60 tiempos individuales están en `results/computer_1/timings_i.csv`.

(j) Comparación entre computadores

Este inciso requiere repetir los experimentos en un segundo computador. Los scripts aceptan `--output-dir results/computer_2` para conservar sus resultados separados. La comparación final debe informar CPU, cores lógicos, RAM, sistema operativo, versiones de Python y NumPy, backend BLAS, tiempos, speedup, eficiencia y mejor combinación (p, t) de ambos equipos. Esta sección queda pendiente hasta disponer de mediciones reales del segundo computador; no es correcto extrapolarlas desde el primero.

Declaración de uso de Inteligencia Artificial

Se utilizó OpenAI Codex como apoyo para revisar y optimizar el código, diseñar los experimentos y redactar el informe. Los integrantes del grupo revisaron el código y validaron experimentalmente los resultados presentados.